

Explainable Machine Learning Framework for Phishing URL Detection with a Realistic Large-Scale Dataset

Elif Ozcelik
Computer Engineering Department
Ankara University
Ankara, Turkey
ozcelikelif22@gmail.com

Farah ElSilk
Computer Engineering Department
Ankara University
Ankara, Turkey
farahselek@gmail.com

Murat Osmanoglu
Computer Engineering Department
Ankara University
Ankara, Turkey
mosmanoglu@ankara.edu.tr

Abstract—Phishing attacks remain a major cybersecurity threat, often evading traditional blacklist-based defenses through continuously evolving URLs. While machine learning (ML) methods achieve high detection performance, their lack of interpretability limits practical adoption. This study proposes an explainable phishing URL detection framework supported by a newly constructed large-scale dataset of 579,920 URLs. Unlike conventional domain-based collections, realistic and structurally diverse legitimate URLs are generated using protocol variations and a two-stage web crawling strategy, reducing dataset bias and improving generalization. Six ML models and three ensemble strategies are evaluated, where LightGBM achieves the best base performance and stacking ensembles provide marginal gains. For interpretability, global explanations are obtained using SHAP and permutation importance, while local decisions are analyzed with SHAP and LIME. Additionally, a SHAP stability analysis based on Spearman correlation demonstrates highly consistent feature importance rankings across different sample sizes. The results show that the proposed framework enables accurate, robust, and reliable explainable phishing detection for real-world applications.

Index Terms—phishing detection, machine learning, explainable artificial intelligence, ensemble learning

I. INTRODUCTION

The rapid integration of digital technologies has significantly increased users' exposure to cybersecurity threats. Among these threats, phishing remains one of the most prevalent attack techniques. In a phishing attack, adversaries craft deceptive URLs that closely mimic legitimate websites, luring users into entering sensitive information such as login credentials, banking details, or personal identification data. Unlike many other cyberattacks that target system vulnerabilities, phishing exploits human trust, making it effective regardless of the technical sophistication of the victim.

Traditional blacklist-based defenses often fail against rapidly evolving attack patterns, as attackers continuously generate new and previously unseen URLs. Consequently, machine learning-based methods have gained prominence due to their ability to learn complex patterns and detect previously unseen threats, demonstrating strong performance in learning discriminative patterns from URL lexical features [2], [4]. However, the black-box nature of high-performing machine learning models limits their applicability in security-critical environments where transparency and trust are essential. Explainable artificial intelligence (XAI) techniques, such as SHAP and LIME, have been widely used to provide insights into model decisions [24], [25]. Nevertheless, their reliability is not always systematically evaluated. In particular,

the stability of explanation methods across different data samples remains largely unexplored.

Furthermore, many existing studies rely on outdated datasets or focus primarily on lexical features, often leading to potential bias where models depend on superficial characteristics such as URL length. This limitation reduces the generalization capability of models when applied to real-world data [26]. In addition, legitimate URLs are often derived from domain lists, which may result in structurally simpler representations compared to phishing URLs.

To address these limitations, this study constructs a new large-scale and up-to-date phishing URL dataset comprising 579,920 URLs. Unlike conventional approaches that rely on domain-only sources for legitimate samples, we generate structurally diverse and realistic URLs by combining protocol and subdomain variations with a two-stage web crawling strategy that captures both shallow and deep links. This process produces longer and more complex legitimate URLs, reducing dataset-induced bias and improving model generalization. On this dataset, six machine learning models and three ensemble strategies are evaluated. For interpretability, we perform global explanation using SHAP and permutation importance, and analyze instance-level decisions through SHAP and LIME. In addition, a SHAP stability analysis based on Spearman rank correlation is conducted to assess the robustness of feature importance rankings across varying sample sizes. Experimental results demonstrate that the proposed framework achieves strong detection performance while providing consistent and reliable explanations.

II. RELATED WORK

Phishing detection has evolved from static methods toward [1] intelligent, data-driven approaches, as attackers became increasingly capable of bypassing rule-based systems. This shift encouraged the integration of AI to improve detection performance, adaptability, and generalization. One of the earliest influential studies in this direction was conducted by Ma et al. [2], who showed that lexical and host-based URL features alone could help to effectively identify malicious websites. Their work laid a foundation for machine learning in phishing detection and motivated further research into which feature sets and models perform best.

Following this early progress, Sahingoz et al. [3] evaluated seven classifiers on 73,000 URLs using lexical, NLP-based, and hybrid features, reporting that Random Forest achieved

very high accuracy. Similarly, Abutaha et al. [4] developed a phishing detection model using several machine learning algorithms, such as SVM, Neural Networks, Gradient Boosting, and Random Forest, that employs 22 lexical features. They evaluated the performance of the model using a dataset containing more than one million URLs. Jalil et al. [5] further refined this direction through feature selection, reducing 100 features to 30 high-impact ones across six datasets, with Random Forest again emerging as the most consistently effective model. In parallel, gradient boosting approaches became increasingly prominent; Odeh et al. [6] reported that CatBoost outperformed XGBoost and LightGBM in their experiments on a dataset of 522,214 URLs. This stage of the literature reflects a shift from comparing traditional ML models and handcrafted features toward more advanced architectures.

Building on the success of strong individual classifiers, researchers then explored ensemble learning strategies to improve performance further. Karim et al. [7] combined Logistic Regression, SVM, and Decision Tree models through voting mechanisms, reaching a remarkable accuracy after hyperparameter tuning and cross-validation. Banik and Sarma [8], adopted a sequence-based deep learning approach, showing that stacked and bagged LSTM ensembles trained directly on raw URL characters achieved high accuracy on 247,000 URLs while keeping the false positive rate very low, outperforming the traditional machine learning baselines in their study.

Despite high predictive performance, many phishing detection models remain difficult to interpret, limiting trust and practical deployment. This has increased interest in explainable artificial intelligence techniques. Pavani et al. [10] addressed this issue by applying SHAP and LIME to a Random Forest model, enabling feature-level explanations and identifying the most influential URL characteristics. In a similar direction, Alam et al. [9] proposed an explainable ensemble framework based on stacking, hard voting, and soft voting, interpreted through SHAP, LIME, and Partial Dependence Plots, achieving a very high recall. Other studies extended explainability from interpretation to feature optimization. Shafin [11] demonstrated that integrating SHAP and LIME into the feature selection process enhances Random Forest performance by emphasizing the most informative features. Likewise, Alsquayh et al. [12] designed a hybrid feature engineering pipeline combining URL-based and HTML-based features with SHAP-based explanations to improve both predictive performance and transparency.

Although these studies demonstrate substantial progress, important gaps remain. In particular, the stability of SHAP explanations across different sample sizes has not been sufficiently validated, and only a limited number of studies combine lexical, DNS, and WHOIS features within a single large-scale experimental framework. Our study addresses these limitations by constructing an up-to-date dataset of 579,920 URLs and evaluating six machine learning models together with three ensemble strategies.

III. MATERIALS AND METHODS

In this section, we describe the dataset construction process and the feature extraction pipeline, and review the methods employed to develop our phishing URL detection model.

A. Dataset Creation

The effectiveness of machine learning in phishing detection depends heavily on dataset quality. Given the short lifespan and rapid turnover of phishing attacks, up-to-date and representative data are essential for developing robust and generalizable models [1]. To address this, we construct a new dataset that captures both recent phishing attacks and the structural diversity of realistic web URLs.

We collected phishing URLs from publicly available threat intelligence platforms, including PhishTank, OpenPhish, and PhishStats. To ensure that the dataset reflects recent attack patterns, we only considered phishing URLs reported from November 2023 to November 2025. We then applied a data cleaning process, removing inaccessible, invalid, or erroneous records. On the other hand, we formed the legitimate part of our dataset using the domain lists obtained from the Tranco ranking and the SimilarWeb platform. The Tranco list provides a ranking of the most visited websites globally, whereas SimilarWeb offers regional web traffic analysis. To represent regional internet usage patterns, we used a list of the top 10,000 most visited domains in Turkey obtained from SimilarWeb using the publicly available data provided by its free access tier. It is worth noting that legitimate domain names were obtained using updated lists corresponding to November 2025.

Since these sources primarily provided domain names, they tend to be significantly shorter and structurally simpler than phishing URLs. Training a model directly on such entries could mislead the classifier and result in false positives for longer legitimate URLs. Therefore, we generated URL variations for each domain using different protocols and subdomain combinations to produce realistic URL examples. We subsequently expanded these basic URLs using a specially developed web crawling mechanism following a two-tiered strategy, as illustrated in Fig. 1. In the first stage, we extracted up to five first-level internal links from each root URL. In the second stage, we collected up to eight deeper links from each of these first-level pages. This approach ensured that the dataset contained not only main page URLs but also longer and structurally more complex URL examples. To comply with robots.txt rules, which define access policies for automated web crawlers, and to prevent excessive load on the target servers, we limited the number of URLs collected for each domain.

We applied a data cleaning process to the collected URLs. During this stage, we removed duplicate records and retained only accessible URLs returning valid HTTP response codes in the 200–399 range. After completing these preprocessing steps, the dataset contained a total of 579,920 URL samples. Of these, 58.5% correspond to legitimate URLs (339,074) and 41.5% correspond to phishing URLs (240,846). To ensure

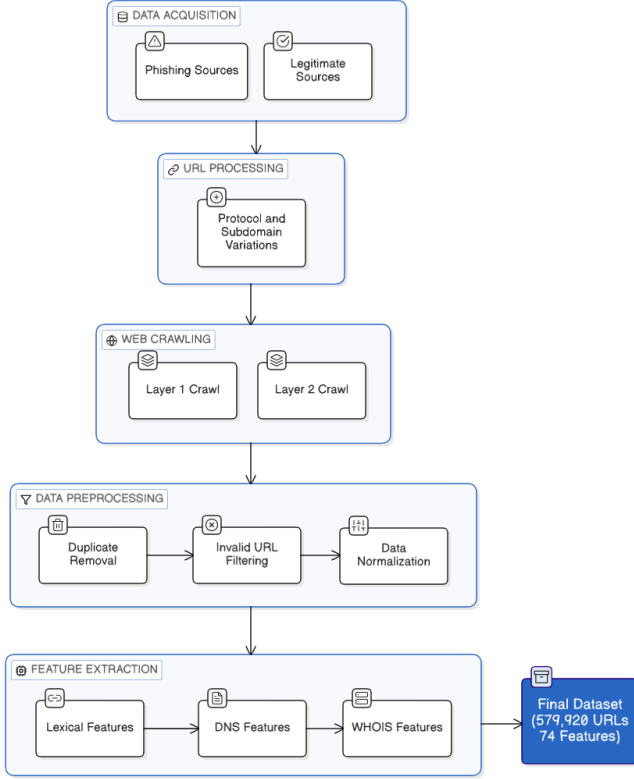


Fig. 1. Dataset creation pipeline.

reproducibility and transparency, the dataset constructed in this study has been publicly released [30].

Although the dataset exhibits a moderate class imbalance, additional experiments were conducted using class balancing techniques to evaluate the potential impact of class distribution on model performance. These experiments did not produce significant differences in the evaluation metrics. Therefore, the original dataset distribution was preserved in the final experiments.

Note that the dataset construction and feature extraction pipeline was implemented in Python using standard libraries for web requests, HTML parsing, DNS querying, WHOIS data collection, and URL parsing.

B. Feature Engineering

In this study, we employ a total of 74 numerical features that represent both the structural and infrastructural characteristics of URLs. These features are grouped into three main categories: lexical features, DNS-based features, and WHOIS-based features.

Lexical Features: Lexical features are derived directly from the URL string and capture its structural properties, such as length, character composition, and the presence of suspicious patterns. Since phishing URLs are often crafted to mimic legitimate ones through subtle manipulations, lexical analysis can reveal irregularities that are difficult to detect visually. In this study, 53 lexical features are extracted, including URL length, hostname length, subdomain count, special character density, digit ratio, and entropy.

DNS-Based Features: DNS-based features provide information about domain infrastructure by analyzing DNS records. Phishing domains often exhibit distinct DNS characteristics, such as low TTL values indicating short-lived infrastructure or missing mail exchange records. We extract 9 such features, including TTL values, MX records, and other DNS configuration parameters.

WHOIS-Based Features: WHOIS-based features describe domain registration characteristics and lifecycle information. Since phishing attacks frequently rely on newly registered domains with short lifespans, registration metadata serves as a strong indicator for distinguishing phishing URLs from legitimate ones. We extract 12 such features, including domain age, registration period, and related metadata.

C. Methods

Logistic Regression (LR): The method is used as a linear baseline model for binary classification. It provides a simple and interpretable approach for establishing a linear decision boundary in the feature space. The probability of the positive class is computed using the sigmoid function as follows [13]:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (1)$$

Decision Tree (DT): It is a non-linear classification model that recursively partitions the feature space into hierarchical decision rules based on impurity criteria. Despite its interpretability, tree depth must be controlled through pruning to prevent overfitting on high-variance feature sets [14].

Random Forest (RF): The method is an ensemble method that trains multiple decision trees on bootstrap samples and aggregates their predictions to reduce variance and improve generalization. Random feature selection at each stage of tree construction reduces the correlation between decision trees, thereby improving model accuracy and mitigating overfitting [15].

XGBoost: The method builds an ensemble of trees sequentially, where each new tree corrects the residual errors of previous iterations using both first- and second-order gradient information. Regularization techniques and efficient tree pruning prevent overfitting, making it highly effective for large-scale structured datasets [16]:

$$Obj(\theta) = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k) \quad (2)$$

LightGBM: It implements gradient boosting using a histogram-based algorithm that grows trees in a leaf-wise manner rather than the conventional level-wise strategy, reducing loss more efficiently while significantly accelerating training. This design lowers memory consumption and improves computational efficiency, making it suitable for large-scale datasets [17]. Therefore, it is particularly suitable for the dataset used in this study.

CatBoost: The method employs ordered boosting to reduce prediction shift during training and builds symmetric trees to enhance model stability. These design choices reduce the risk

of overfitting and allow CatBoost to deliver consistent performance across diverse datasets without extensive preprocessing [18].

Ensemble Learning Strategies: To further enhance classification robustness and reduce variance, we combine the outputs of the previously described base models using three distinct ensemble strategies:

- *Hard Voting:* In this strategy, each base classifier independently predicts a class label, and the final output is determined by the majority vote. This approach treats each individual model with equal weight.
- *Soft Voting:* Unlike hard voting, soft voting aggregates the probability distributions produced by each base model. The final prediction is based on the highest average probability across all models. This allows the ensemble to incorporate the confidence levels of the individual classifiers into the decision-making process.
- *Stacking:* This strategy employs a multi-level learning architecture. Base model predictions serve as input features for a second-level meta-classifier. To prevent data leakage, out-of-fold predictions generated through cross-validation are used to construct the meta-level training dataset. This ensures that the meta-classifier is trained solely on predictions for unseen instances, enabling the model to leverage the strengths of individual classifiers while mitigating their weaknesses [19].

SHapley Additive exPlanations (SHAP): SHAP quantifies each feature’s contribution to model predictions using Shapley values, providing a theoretically consistent measure of feature importance. For tree-based models, the TreeSHAP algorithm computes exact SHAP values efficiently by leveraging the hierarchical structure of decision trees [20]:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (3)$$

Global SHAP explanations provide interpretable feature importance scores across the entire dataset, while local SHAP explanations decompose individual predictions into feature-level contributions, revealing how each feature pushes a specific prediction toward or away from a particular class [20].

Permutation Importance: This method evaluates feature importance by randomly shuffling feature values and measuring the resulting decrease in model performance. A larger performance drop indicates higher importance. It provides a simple, model-agnostic global measure of feature importance that can be used together with other learning methods to assess feature relevance [21].

Local Interpretable Model-agnostic Explanations (LIME): LIME approximates the behavior of a complex model around a specific instance using a locally weighted linear model. It perturbs input data and observes changes in predictions to identify influential features. This approach provides interpretable, model-agnostic local explanations [22].

IV. PROPOSED METHOD

This section presents the experimental setup, system architecture, and evaluation methodology used in this study.

A. Experimental Setup

All experiments were carried out on a system equipped with an 11th-generation Intel Core i7-11800H processor, featuring 8 physical cores and 16 logical threads, along with 16 GB of RAM. We implemented the models in a Python-based Jupyter Notebook environment. The study uses Pandas and NumPy for data preprocessing, Scikit-learn for building and evaluating machine learning models, SHAP and LIME for explainability, and Matplotlib for visualization. In addition, we used python-whois and dnspython to extract WHOIS and DNS-related features.

B. System Architecture

The proposed system consists of a multi-stage architecture designed to detect phishing URLs and provide explainable model decisions. The system includes five main stages: data preparation, feature extraction, model training, ensemble learning, and explainability analysis.

1) Data Preparation: We collect and prepare the dataset by gathering raw URL data and applying a series of preprocessing steps to ensure data quality and accessibility. This process includes removing invalid or unreachable URLs, handling missing or inconsistent entries, and organizing the data into a structured format suitable for analysis. Following preprocessing, the dataset is divided into training and testing subsets using an 80%–20% split with a fixed random seed (42) to ensure reproducibility.

2) Feature Extraction: We extract a comprehensive set of features to represent the characteristics of each URL. These features include lexical attributes derived from the URL structure, as well as infrastructural information obtained from DNS and WHOIS data. This combination enables the model to capture both surface-level patterns and underlying domain-related properties associated with phishing behavior.

3) Model Training: We use the extracted features to train multiple machine learning models that learn to distinguish between phishing and legitimate URLs. Each model is trained under the same experimental conditions to ensure a fair comparison. Hyperparameter values were determined based on commonly used configurations. We then applied a limited tuning stage to examine whether further adjustment would lead to meaningful improvement. Since the resulting gains were only minor, the final settings were retained to preserve a practical balance between detection performance and computational cost.

We trained the Logistic Regression model using the LBFGS solver with a maximum of 1000 iterations. The Decision Tree model was constructed based on the Gini impurity criterion, while the Random Forest model was configured with 200 decision trees. For the gradient boosting methods, we trained the XGBoost model with 300 trees, a learning rate of 0.05, and a maximum depth of 6. The LightGBM model was configured with 300 trees, a learning rate of 0.05, and 31 leaves

(num_leaves = 31), whereas we trained the CatBoost model using 300 iterations, a depth of 6, and a learning rate of 0.05. All models were trained on the same training dataset and evaluated on the same test dataset to ensure a fair comparison across algorithms.

4) Ensemble Learning: We apply ensemble learning techniques to integrate the predictions of the trained base models, aiming to enhance predictive performance and improve model robustness by leveraging the complementary strengths of different classifiers. In this study, ensemble models were constructed using the predictions of the six base classifiers described in Section III-C. Three ensemble strategies were implemented: hard voting, soft voting, and stacking.

In the stacking ensemble approach, the predictions of the base models were used as input features for a meta-model. To avoid data leakage in the stacking framework, base models were trained using 5-fold cross-validation. At each fold, the models generated probability outputs for the held-out instances, and these out-of-fold probabilities were directly concatenated across base models to construct the meta-level training dataset. This resulted in a feature matrix of size $n \times m$, where n is the number of training instances and m is the number of base models.

A Logistic Regression model was used as the meta-model to combine the outputs of the base classifiers. The meta-model learned how to integrate the prediction patterns produced by different base models in order to generate the final prediction. Ensemble models were evaluated under two different configurations. In the first configuration, the ensemble method was applied using all base models. In the second configuration, the ensemble was applied only to the three models that provided the best trade-off between predictive performance and computational cost, namely LightGBM, CatBoost, and Random Forest.

5) Explainability Analysis: Finally, we apply explainability methods to interpret the decision-making behavior of the models and to provide transparency into how predictions are generated. These methods aim to identify the most influential features and to understand how different inputs contribute to model outputs. Both global and local explainability techniques are employed to analyze model behavior at different levels. Global analysis focuses on determining overall feature importance across the entire dataset, while local analysis provides instance-level explanations that clarify individual predictions. This approach enables a comprehensive understanding of the model, supporting both performance validation and trust in the system.

SHAP analyses were performed using the TreeSHAP algorithm, which enables efficient and exact computation of SHAP values for tree-based models by utilizing the internal structure of decision trees. Permutation importance and SHAP analyses were conducted to determine global feature importance, both computed on the entire test set to capture the overall behavior of the model. Local explainability analyses were conducted using SHAP-based instance-level explanations and the LIME method. For local explanation analysis, 400 samples were

randomly selected from the test set while preserving the class distribution between phishing and legitimate URLs.

SHAP Stability Analysis: In addition to standard explainability analyses, a SHAP stability analysis was conducted to evaluate the robustness of feature importance rankings obtained from SHAP values. In this analysis, SHAP calculations were repeated using subsets of different sizes in order to examine how feature importance rankings change with varying sample sizes. Subsets consisting of 200, 500, 1000, and 1500 samples were used in the experiments, along with the full dataset.

For each subset size, we recalculated SHAP values and obtained the corresponding feature importance rankings. We then evaluated the consistency of these rankings using the Spearman rank correlation coefficient to assess the stability of SHAP explanations across different sample sizes. This analysis provides insight into the robustness and reliability of the interpretability results.

C. Evaluation metrics

The models are evaluated using accuracy, precision, recall, F1-score, and ROC-AUC:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (7)$$

where TP , TN , FP , and FN denote confusion matrix elements. In addition to threshold-based metrics, ROC-AUC is used to evaluate the model's ability to distinguish between classes across different thresholds, where higher values indicate better discriminative performance.

V. RESULTS

This section presents the experimental results obtained from the evaluation of base machine learning models, ensemble learning strategies, and explainable AI analyses.

A. Base Model Performance Analysis

We employed multiple metrics, including accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC, to evaluate the performance of our model. However, the comparative evaluation primarily focuses on F1-score, recall, training time, and test time in order to assess both detection capability and computational efficiency. Recall is particularly important in phishing detection tasks because it reflects the ability of a model to correctly identify phishing websites. Missing phishing URLs may expose users to security risks; therefore, models with higher recall values are preferred. In addition, the F1-score provides a balanced measure of precision and recall, allowing an overall assessment of classification performance.

As shown in Table I, among the evaluated models, LightGBM achieved the highest performance, obtaining an F1-score of 0.9840 and a recall value of 0.9783, demonstrating strong capability in detecting phishing websites. Other models such as

TABLE I
BASE MODEL PERFORMANCE ANALYSIS

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	Train (s)	Test (s)
Logistic Regression	0.9440	0.9439	0.9198	0.9317	0.9845	0.9804	17.27	0.52
Decision Tree	0.9749	0.9721	0.9673	0.9697	0.9738	0.9539	25.54	0.40
Random Forest	0.9851	0.9905	0.9736	0.9820	0.9983	0.9979	142.80	4.01
XGBoost	0.9816	0.9844	0.9711	0.9777	0.9976	0.9972	30.23	1.06
LightGBM	0.9868	0.9898	0.9783	0.9840	0.9985	0.9982	33.83	3.62
CatBoost	0.9832	0.9863	0.9730	0.9796	0.9979	0.9976	102.19	0.48

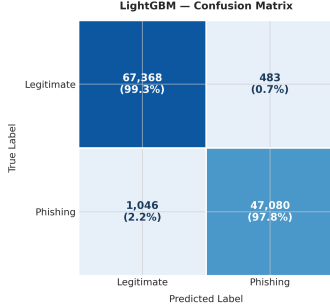


Fig. 2. Confusion matrix of the LightGBM model.

Random Forest, XGBoost, and CatBoost also produced strong classification results; however, differences can be observed in terms of computational cost. For instance, although the Random Forest model achieved an F1-score of 0.9820, it required a significantly longer training time compared to the LightGBM model. When both classification performance and computational efficiency are considered together, LightGBM provides the most balanced performance among the evaluated base models.

On the other hand, as seen in Fig. 2, LightGBM demonstrated exceptional performance with a false negative rate of only 2.17% and a very small false positive rate of 0.71%. These results indicate that the model successfully identifies the vast majority of phishing websites while keeping the misclassification of legitimate URLs at a minimal level. Notably, the low false negative rate is particularly critical from a cybersecurity perspective, as undetected phishing URLs pose direct risks to end users.

B. Ensemble Model Performance Analysis

As shown in Table II, stacking-based models achieved the highest classification performance. In particular, the EnsembleStacking_AllBase model achieved the best performance, obtaining an F1-score of 0.9861 and a recall value of 0.9816. However, this improvement in classification performance was obtained at the cost of significantly higher training time. On the other hand, soft voting ensemble models provide a more balanced trade-off between performance and computational cost. For example, the EnsembleSoft_Top3 model achieved an F1-score of 0.9834, while requiring substantially lower training time compared to the stacking-based models.

As seen in Fig. 3, the EnsembleStacking_AllBase model achieved a small false negative rate of 1.84% and a small false positive rate of 0.62%. The reduction in false negatives is particularly noteworthy, as it indicates that the stacking-based ensemble captures more phishing URLs than individual

TABLE II
ENSEMBLE MODEL PERFORMANCE ANALYSIS

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	Train (s)	Test (s)
EnsembleStacking_AllBase	0.9885	0.9906	0.9816	0.9861	0.9985	0.9982	1455.83	10.80
EnsembleStacking_Top3	0.9877	0.9895	0.9806	0.9851	0.9986	0.9982	1248.78	8.24
EnsembleSoft_Top3	0.9863	0.9904	0.9765	0.9834	0.9985	0.9983	261.36	10.55
EnsembleHard_Top3	0.9861	0.9901	0.9763	0.9832	N/A	N/A	265.45	8.19
EnsembleSoft_AllBase	0.9851	0.9888	0.9752	0.9820	0.9981	0.9978	328.00	11.09
EnsembleHard_AllBase	0.9845	0.9910	0.9714	0.9811	N/A	N/A	272.58	4.99

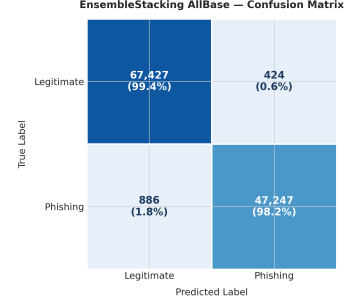


Fig. 3. Confusion matrix of the EnsembleStacking_AllBase model.

base models may miss. These values also represent a modest but meaningful improvement over LightGBM.

When the base models and ensemble models are evaluated together, the results indicate that ensemble methods provide a modest improvement in classification performance. However, this improvement is accompanied by a substantially higher computational cost.

C. Global Feature Importance Analysis

In this study, we conducted global explainability analysis using SHAP and permutation importance methods to understand which features the model relies on in detecting phishing URLs. Among all methods, we selected LightGBM as the primary model for the explainable artificial intelligence analysis, as it provides a strong balance between classification performance and computational efficiency.

1) Global SHAP Analysis: As demonstrated in Fig. 4, which displays the top 20 most influential features obtained as a result of the global SHAP analysis conducted on the full dataset for the LightGBM model, the model's predictions are largely influenced by URL structural properties and domain-related characteristics. Among the most influential features, `hostname_length`, `domain_age_days`, `has_www`, and `entropy_path` contribute strongly to the model output. The wide distribution of SHAP values across these features indicates that phishing URL structures can vary significantly and that the model captures diverse structural patterns associated with phishing attacks.

2) Permutation Importance Analysis: We performed permutation importance analysis to evaluate the contribution of individual features to the model's overall detection capability. As shown in Fig. 5, which displays the top 20 most influential features obtained from the permutation importance analysis conducted on the full dataset for the LightGBM model, URL structural characteristics and domain infrastructure features have a significant impact on model performance. In particular, features such

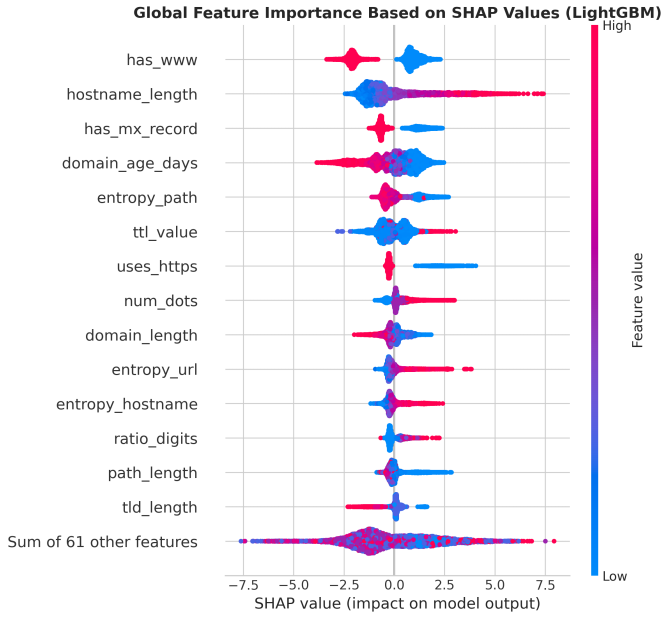


Fig. 4. SHAP summary plot for the LightGBM model.

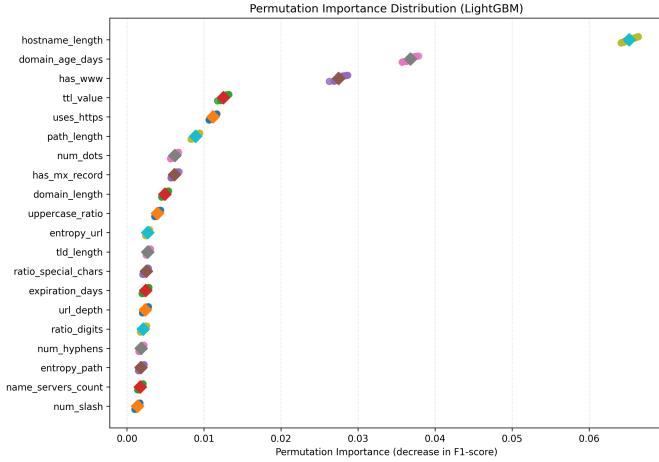


Fig. 5. Permutation importance distribution for the LightGBM model.

as `hostname_length`, `domain_age_days`, `has_www`, `ttl_value`, and `domain_length` play a critical role in distinguishing phishing websites from legitimate ones.

3) Consistency Between SHAP and Permutation Importance: To further evaluate the stability of the global explanation results, we analyzed the relationship between SHAP-based feature rankings and permutation importance rankings for the LightGBM model. As shown in Table III, a strong agreement was observed between the SHAP and permutation importance rankings. The Spearman rank correlation coefficient of $\rho = 0.916$ indicates that both explainability approaches identify largely similar influential features. Besides, several structural URL features consistently appear among the most influential predictors in both methods. In particular, `hostname_length`, `domain_age_days`, `num_dots`, and `domain_length` appear among the top-ranked features across both explainability techniques. Additionally, domain and DNS-related features such as `has_www`,

TABLE III
SPEARMAN RANK CORRELATION BETWEEN SHAP AND PERMUTATION IMPORTANCE

Model	Features Compared	Spearman ρ	p-value	Top-10 Feature Overlap	Top-20 Feature Overlap
LightGBM	74	0.916	1.04×10^{-30}	8	17

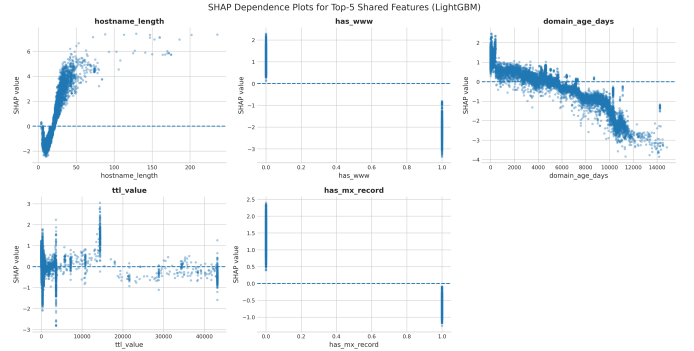


Fig. 6. SHAP dependence plots for the top-5 shared features identified by SHAP and permutation importance.

`ttl_value`, and `has_mx_record` also emerge as important indicators.

These findings are consistent with known phishing strategies in which attackers manipulate URL structures and frequently rely on newly registered domains or complex hostname patterns to deceive users. This alignment suggests that the model captures meaningful behavioral patterns associated with real-world phishing attacks.

4) Feature Behavior Analysis: To examine how the most influential features affect the model predictions, we generated SHAP dependence plots for the top five shared features identified by both SHAP and permutation importance. As shown in Fig. 6, increasing values of `hostname_length` strengthen the model's phishing prediction. This pattern is consistent with phishing strategies in which attackers construct longer and more complex domain structures in order to mimic legitimate websites and mislead users.

In contrast, the `domain_age_days` feature shows an inverse relationship with phishing predictions. Newly registered domains contribute more strongly to phishing classifications, whereas older domains are more likely to be associated with legitimate websites. This observation is consistent with the findings of Lee et al. [23], who reported that phishing URLs have an average lifespan of approximately 54 hours, with a median of only 5.46 hours, highlighting the tendency of phishing infrastructures to rely on recently registered, short-lived domains.

DNS-related features also play an important role in the model's decision-making process. Features such as `ttl_value` and `has_mx_record` indicate that the model relies not only on lexical URL patterns but also on operational signals related to domain infrastructure.

These findings suggest that the model captures both structural URL characteristics and domain-level infrastructure signals, which together provide meaningful indicators for distinguishing phishing websites from legitimate ones.

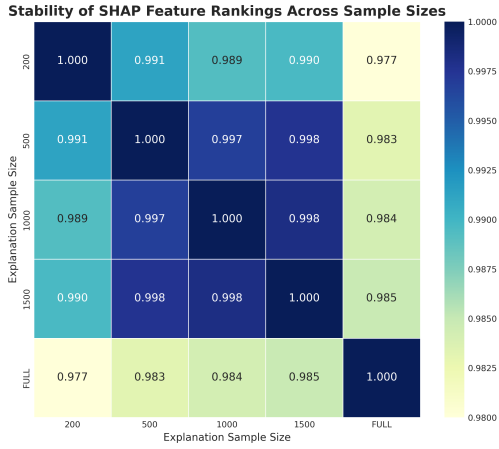


Fig. 7. Spearman rank correlation matrix of SHAP feature rankings across different sample sizes.

D. Incremental Global SHAP Analysis

To evaluate the reliability of the explainability results, we computed SHAP feature rankings using different sample sizes (200, 500, 1000, and 1500 instances) and compared them with the rankings obtained from the full test dataset using Spearman rank correlation. This analysis aims to assess the extent to which SHAP-based explanations remain consistent across different data sample sizes.

The correlation matrix presented in Fig. 7 shows a high level of consistency between the feature rankings obtained from different sample sizes. In particular, after 500 samples, the correlation values reach approximately 0.997–0.998, indicating that the global feature importance rankings remain highly stable as the sample size increases. Even when using only 200 samples, the correlation with the full test dataset remains relatively high at 0.977, suggesting that the overall feature ranking structure is largely preserved even with relatively small sample sizes.

Fig. 8 illustrates the rank changes of the top 10 most important features across different sample sizes. The results show that features such as `hostname_length`, `domain_age_days`, and `ttl_value` remain consistently ranked among the most influential predictors across all sample sizes.

These findings demonstrate that the SHAP explanations are highly robust to sampling variations, indicating that the identified feature importance patterns remain stable even when the explanation analysis is performed on smaller subsets of data.

E. Local Explanation Analysis

To examine the model’s instance-level decision behavior, we generated local explanations using SHAP and LIME methods on a total of 400 samples.

As shown in Fig. 9, features such as `has_www`, `hostname_length`, `entropy_path`, and `domain_age_days` frequently influence individual predictions. These findings are consistent with the global

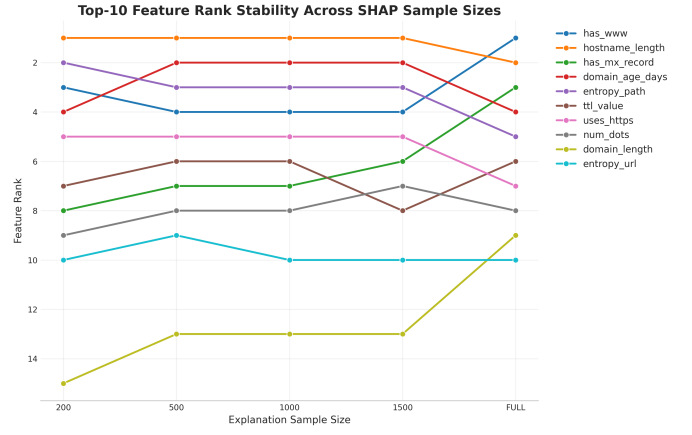


Fig. 8. Top-10 feature rank stability across SHAP sample sizes.

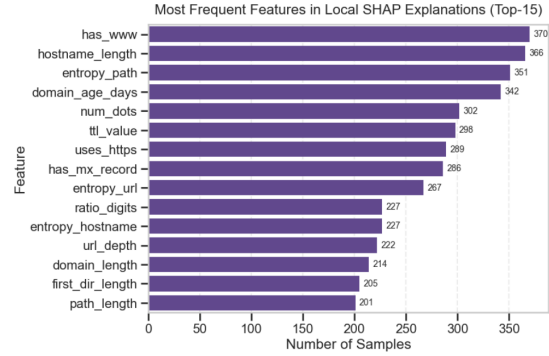


Fig. 9. Most frequent features in local SHAP explanations.

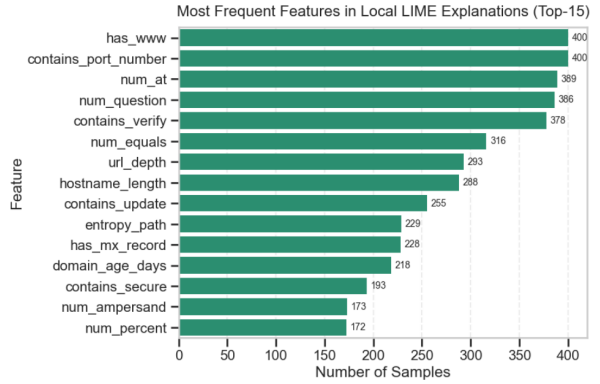


Fig. 10. Most frequent features in LIME explanations.

importance analysis, suggesting that the model relies on similar decision signals at both the global and local levels.

In contrast, Fig. 10 reveals that LIME highlights structural anomalies within URL patterns, including features such as `contains_port_number`, `num_at`, and `num_question`. These characteristics represent suspicious elements commonly observed in phishing URLs.

Comparing the two methods, we observed that several features are consistently emphasized by both SHAP and LIME. In particular, `has_www`, `hostname_length`, `domain_age_days`, and `entropy_path` emerge as important predictors across both approaches, indicating that the model strongly relies on specific structural URL characteristics

when identifying phishing websites. However, while LIME tends to emphasize instance-specific local signals, SHAP provides a more consistent representation of the model’s overall decision behavior. The combined use of these two explanation techniques therefore enables a more comprehensive understanding of model behavior at both the global and local levels.

VI. DISCUSSION

This section analyzes the experimental findings from multiple perspectives, including model performance, alignment with prior studies, XAI insights, and the practical strengths and limitations of the proposed framework.

Classification Results: The results indicate that phishing URL detection involves a nonlinear and high-dimensional feature space. Accordingly, gradient boosting models outperformed the linear baseline and the single-tree model in this study. Among the evaluated base models, LightGBM achieved the highest F1-score (0.9840) while also maintaining a favorable balance between predictive performance and computational efficiency. This result suggests that boosting-based methods are particularly well suited for structured tabular phishing data. In contrast, the comparatively lower performance of the Decision Tree model indicates that single-tree learners may be more prone to overfitting when trained on high-variance lexical, DNS, and WHOIS-based features.

Stacking-based ensemble models provided only a modest improvement in classification performance ($F1 = 0.9861$). This outcome is expected because the base models already achieved high and relatively similar performance levels, leaving limited opportunity for the meta-learner to exploit systematic disagreements.

Literature Consistency: The features identified as most influential in this study are largely consistent with those reported in the phishing detection literature. In particular, structural URL characteristics such as `hostname_length`, `domain_length`, and `num_dots` have been reported as useful indicators in phishing detection studies. Akhtar et al. [29] observed that lexical features, including `hostname` and `domain length`, tend to play an important role in distinguishing phishing URLs across multiple datasets. Similarly, Shafin [11] showed that features identified as important by SHAP are further validated through LIME, highlighting their consistent importance across both global and local perspectives.

The importance of `domain_age_days` is likewise consistent with prior work. Sumner et al. [27] demonstrated that domain age is the most significant feature for phishing detection, noting that phishing domains are typically newly registered and have shorter operational lifespans. This finding is further supported by Kustiawan et al. [28], who, through a systematic review of recent studies, identified WHOIS-derived features, including domain age, as consistently reported high-impact attributes in phishing detection and feature engineering research.

Furthermore, the strong agreement observed between SHAP and permutation importance rankings (Spearman $\rho = 0.916$)

indicates that different explainability techniques consistently identify similar influential features.

XAI Insights: The explainability analysis provides deeper insight into the decision behavior of the model. Global SHAP and permutation importance analyses reveal that the model relies heavily on structural URL complexity and domain life-cycle characteristics. Features such as `hostname_length`, `domain_age_days`, and `entropy_path` were consistently identified as important predictors for phishing detection.

Local explanation analyses further illustrate how these features influence individual predictions. SHAP-based local explanations show that the features highlighted in global analyses also frequently contribute to instance-level predictions, indicating consistent decision patterns across the dataset. In contrast, LIME explanations tend to emphasize local irregularities within URL structures. Features such as `contains_port_number`, `num_at`, and `num_question` were more frequently highlighted in LIME explanations, representing suspicious characters and structural anomalies commonly observed in phishing URLs. These observations suggest that SHAP captures globally consistent decision signals, while LIME provides complementary insights into localized anomalies affecting individual predictions.

Framework Strengths and Limitations: The proposed framework has several notable strengths. First, the combination of lexical, DNS, and WHOIS-based features provides a multi-dimensional representation that captures both URL structure and domain infrastructure characteristics. This representation enables robust phishing detection across a large and diverse dataset consisting of 579,920 URLs. Second, the LightGBM model offers a strong balance between predictive performance and computational efficiency. Compared to more complex ensemble approaches such as stacking, LightGBM achieves similar performance while maintaining lower training and inference costs, making it suitable for near-real-time detection systems. Additionally, the SHAP stability analysis conducted in this study demonstrates that feature importance rankings remain highly consistent across different sample sizes. Reliable explanations can be obtained even from subsets as small as 500 samples ($\rho \geq 0.997$), suggesting that explainability analyses can remain stable without requiring full-dataset computations.

However, the framework also has certain limitations. Feature extraction involving DNS queries and WHOIS lookups introduces additional latency, which may limit the feasibility of real-time browser-level detection systems. Furthermore, since the model relies primarily on URL structural characteristics and domain metadata, it may struggle to detect phishing attacks hosted on legitimate infrastructures such as cloud services or URL shortening platforms.

VII. CONCLUSION

This study presents a comprehensive phishing URL detection framework that integrates machine learning and explainable artificial intelligence techniques. A large-scale dataset was constructed using a multi-stage web crawling approach to

generate realistic and structurally diverse URL samples, and a diverse set of lexical, DNS-based, and WHOIS-based features was utilized to capture both structural and infrastructural characteristics of URLs.

Experimental results demonstrate that gradient boosting models, particularly LightGBM, achieve the best balance between detection performance and computational efficiency, reaching an F1-score of 0.9840, while ensemble methods provide only marginal improvements at higher computational cost. Global explainability analyses using SHAP and permutation importance show strong agreement ($\rho = 0.916$), consistently highlighting hostname length, domain age, and DNS-related features as the most influential predictors.

In addition to performance evaluation, this study introduces a SHAP stability analysis to assess the consistency of feature importance rankings across different sample sizes. The results indicate that SHAP-based explanations remain highly stable, supporting the reliability of the interpretability findings.

REFERENCES

- [1] S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, and C. Zhang, "An empirical analysis of phishing blacklists," in *Proc. 6th Conference on Email and Anti-Spam (CEAS)*, Mountain View, CA, USA, 2009.
- [2] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: learning to detect malicious web sites from suspicious URLs," in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Paris, France, Jun. 2009, pp. 1245–1254.
- [3] E. K. Sahingoz, B. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Syst. Appl.*, vol. 117, pp. 345–357, 2019.
- [4] M. Abutaha, M. Ababneh, K. Mahmoud, and S. A. H. Baddar, "URL phishing detection using machine learning techniques based on URLs lexical analysis," in *Proc. 2021 12th Int. Conf. Information and Communication Systems (ICICS)*, Amman, Jordan, 2021, pp. 147–152.
- [5] S. Jalil, M. Usman, and A. Fong, "Highly accurate phishing URL detection based on machine learning," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 7, pp. 9233–9251, 2023.
- [6] A. Odeh, Q. Abu Al-Haija, A. Aref, and A. Abu Taleb, "Comparative study of CatBoost, XGBoost, and LightGBM for enhanced URL phishing detection: a performance assessment," *J. Internet Serv. Inf. Secur.*, vol. 13, no. 4, pp. 1–11, 2023.
- [7] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," *IEEE Access*, vol. 11, pp. 36805–36822, 2023.
- [8] B. Banik and A. Sarma, "Phishing URL detection using LSTM based ensemble learning approaches," *Int. J. Comput. Netw. Commun.*, vol. 15, no. 1, pp. 17–33, 2023.
- [9] R. Alam, A. Khune, T. V. Kalal, and A. Nautiyal, "E2Phish: explainable ensemble machine learning model for enhanced phishing URL detection," in *Proc. IEEE 8th Int. Conf. Information and Communication Technology (CICT)*, Prayagraj, India, 2024, pp. 1–6.
- [10] B. V. Pavani, D. Mahitha, and B. Uma Maheswari, "Enhancing online safety: phishing URL detection using machine learning and explainable AI," in *Proc. 15th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, Jun. 2024, pp. 1–6.
- [11] S. S. Shafin, "An explainable feature selection framework for web phishing detection with machine learning," *Data Sci. Manag.*, vol. 8, no. 2, pp. 127–136, 2025.
- [12] N. Alsquayh, A. Mirza, and A. Alhogail, "A phishing website detection system based on hybrid feature engineering with SHAP explainable artificial intelligence technique," in *Proc. Int. Conf. Web Inf. Syst. Eng. (WISE)*, LNCS, vol. 15463, Singapore: Springer, 2025, pp. 3–17.
- [13] J. K. Harris, "Primer on binary logistic regression," *Fam. Med. Community Health*, vol. 9, no. Suppl 1, Art. no. e001290, 2021.
- [14] S. B. Kotsiantis, "Decision trees: a recent overview," *Artif. Intell. Rev.*, vol. 39, no. 4, pp. 261–283, 2013.
- [15] H. A. Salman, A. Kalakech, and A. Steiti, "Random forest algorithm overview," *Babylonian J. Mach. Learn.*, vol. 2024, pp. 69–79, 2024.
- [16] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, "A comparative analysis of XGBoost," 2019, *arXiv:1911.01914*. [Online]. Available: <https://arxiv.org/abs/1911.01914>
- [17] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [18] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [19] P. Mahajan, S. Uddin, F. Hajati, and M. A. Moni, "Ensemble learning for disease prediction: a review," *Healthcare*, vol. 11, no. 12, Art. no. 1808, 2023.
- [20] S. M. Lundberg, G. G. Erion, and S.-I. Lee, "Consistent individualized feature attribution for tree ensembles," 2018, *arXiv:1802.03888*. [Online]. Available: <https://arxiv.org/abs/1802.03888>
- [21] A. Altmann, L. Tološi, O. Sander, and T. Lengauer, "Permutation importance: a corrected feature importance measure," *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, 2010.
- [22] G. Visani, E. Bagli, F. Chesani, A. Poluzzi, and D. Capuzzo, "Statistical stability indices for LIME: obtaining reliable explanations for machine learning models," *J. Oper. Res. Soc.*, vol. 73, no. 1, pp. 91–101, 2022.
- [23] K. Lee, K. Lim, H. Kim, Y. Kwon, and D. Kim, "7 days later: analyzing phishing-site lifespan after detected," in *Proc. ACM Web Conf. (WWW)*, Sydney, Australia, 2025, pp. 945–956.
- [24] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [25] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 1135–1144.
- [26] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, "Phishing URL detection generalisation using unsupervised domain adaptation," *Comput. Netw.*, vol. 245, Art. no. 110398, 2024.
- [27] A. Sumner, J. Xu, and X. Yuan, "Determining phishing emails using URL domain features," in *Proc. 2022 1st Int. Conf. on AI in Cybersecurity (ICAIC)*, Victoria, TX, USA, 2022, pp. 1–5.
- [28] Y. A. Kustiawan and K. I. Ghauth, "Feature engineering for phishing website detection using machine learning: a systematic review," *IEEE Access*, vol. 13, pp. 192083–192100, 2025.
- [29] H. M. U. Akhtar, M. Nauman, N. Akhtar, M. Hameed, S. Hameed, and M. Z. Tareen, "Mitigating cyber threats: machine learning and explainable AI for phishing detection," *VFAST Trans. Softw. Eng.*, vol. 13, no. 2, pp. 170–195, 2025.
- [30] E. Ozcelik, F. ElSilk, and M. Osmanoglu, "Phishing URL features dataset," Kaggle, 2026. [Online]. Available: <https://www.kaggle.com/dsv15339081>, doi: 10.34740/KAGGLE/DSV/15339081.